Security Review

Cantina AI Audit

TELCOIN: TEL-V3

Verified by:
Cergyk, Lead Security Researcher

CANTINA

August 17, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

This AI-driven security review provides an automated evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. AI-based analysis cannot guarantee that every vulnerability will be detected and is not a replacement for human review, continuous security measures, penetration testing, or regular code audits. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during this review and would require a new assessment.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Telcoin (TEL) is the native gas token powering the Telcoin Network, a Layer 1 EVM blockchain where Mobile Network Operators (MNOs) serve as network validators.

From Jul 7th to Jul 14th an AI-driven review was conducted of tel-v3 on commit hash 7728a920. The automated analysis identified a total of **13** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	7	4	3
Gas Optimizations	0	0	0
Informational	6	5	1
Total	**13**	**9**	**4**

2.1 Scope

The security review had the following components in scope for tel-v3 on commit hash 7728a920:

```
src
├── MigrationVault.sol
├── MintBurnWrapper.sol
├── NativeBridge.sol
├── TelcoinBridge.sol
├── TelcoinV3.sol
├── TokenMigration.sol
├── helpers
│   ├── EIP3009.sol
│   └── Roles.sol
├── interfaces
│   ├── IEIP3009.sol
│   ├── IERC20Mintable.sol
│   └── ITelcoinBridge.sol
└── legacy
    └── Telcoin.sol
```

3 Findings

3.1 Low Risk

3.1.1 `NativeBridge` / `TelcoinBridge` API exposes LayerZero-specific parameters without enforcing intended defaults

Severity: Low Risk.

Context: TelcoinBridge.sol#L72-L78.

The only transfer endpoint exposes the complete LayerZero `SendParam` and `MessagingFee` structures:

```
function send(
    SendParam calldata _sendParam,
    MessagingFee calldata _fee,
    address _refundAddress
) external payable override whenNotPaused returns (...) {
    return _send(_sendParam, _fee, _refundAddress);
}
```

This leaks transport-specific details into integrations and accepts fields that project documentation describes as unused, including:

- `_sendParam.composeMsg`
- `_sendParam.oftCmd`
- `_fee.lzTokenFee`

Integrators can therefore submit configurations outside the project's intended basic-transfer profile.

Recommendation: Add a simplified `bridge()` wrapper that constructs `SendParam`, rejects unsupported commands/composition, and applies project-approved defaults. The inherited `send()` may remain available if advanced LayerZero functionality is intentionally supported.

Telcoin Association: Acknowledged.

Cantina Managed: Acknowledged.

3.1.2 LayerZero Compose messages are accepted without a guaranteed compose execution configuration

Severity: Low Risk.

Context: TelcoinBridge.sol#L73.

The bridge accepts arbitrary `composeMsg` data through `SendParam`:

```
function send(
    SendParam calldata _sendParam,
    MessagingFee calldata _fee,
    address _refundAddress
) external payable override whenNotPaused returns (...) {
    return _send(_sendParam, _fee, _refundAddress);
}
```

A nonempty compose payload changes LayerZero's message type from `SEND` (`1`) to `SEND_AND_CALL` (`2`) and causes the destination to schedule `lzCompose` after crediting tokens. The deployment configuration, however, defines and configures only `SEND`:

```
uint16 constant SEND = 1;
//...
```

```
params[updateCount] = EnforcedOptionParam({
    eid: dst.eid,
    msgType: SEND,
    options: desired
});
```

There is no corresponding enforced-options entry for `SEND_AND_CALL`, and `_buildLzReceiveOption()` only encodes `OPTION_TYPE_LZRECEIVE`; it does not add an executor `lzCompose` gas option:

```
function _buildLzReceiveOption(uint128 gas) internal pure returns (bytes memory) {
    return abi.encodePacked(
        uint16(3),
        uint8(1),
        uint16(17),
        uint8(1), // OPTION_TYPE_LZRECEIVE
        gas
    );
}
```

Consequently, composed messages do not inherit the configured minimum receive gas for ordinary `SEND` messages, and no minimum compose gas is enforced. A caller may supply suitable options manually, but malformed or incomplete `SEND_AND_CALL` options can leave the composed call unexecuted after tokens have already been credited, potentially stranding funds in a recipient contract whose workflow depends on composition.

Recommendation: Either reject nonempty `composeMsg`, or add explicit `SEND_AND_CALL` enforced options containing both `lzReceive` and `lzCompose` gas and test the complete compose lifecycle.

Telcoin Association: Fixed in Issue 39.

Cantina Managed: The `send()` endpoint now rejects any nonempty `composeMsg`, preventing `SEND_AND_CALL` messages from being submitted without the required compose gas options.

3.1.3 `NativeBridge` / `TelcoinBridge` `pause` / `unpause` requiring owner role may affect liveness during emergency

Severity: Low Risk.

Context: NativeBridge.sol#L99-L101, TelcoinBridge.sol#L104-L106, TokenMigration.sol#L147-L159.

The bridge owner controls both pausing directions:

```
function pause() external onlyOwner {
    _pause();
}

function unpause() external onlyOwner {
    _unpause();
}
```

The same owner also controls LayerZero configuration inherited from the OApp and token rescue functionality. If the same governance account owns `MintBurnWrapper`, it additionally controls which bridge receives mint/burn authority.

This combines routine emergency operations with high-impact configuration authority.

Recommendation: Introduce separate pauser and unpauser roles. Keep bridge configuration and wrapper ownership behind a multisig and preferably a timelock.

Telcoin Association: Fixed in Issue 38.

Cantina Managed: `NativeBridge`, `TelcoinBridge`, and `TokenMigration` now extend `AccessControlEnumerable` and define dedicated pauser roles, separating pause authority from broader

owner permissions.

3.1.4 Permit compatibility depends on EIP-7702 delegate implementing ERC-1271

Severity: Low Risk.

Context: EIP3009.sol#L188, TelcoinV3.sol#L140.

Permit validation uses `SignatureChecker`:

```
if (!SignatureChecker.isValidSignatureNow(owner_, hash, signature)) {
    revert InvalidSignature();
}
```

`SignatureChecker` uses ECDSA only when `owner_.code.length == 0`. An EIP-7702 delegated account has code, so validation is routed through `IERC1271.isValidSignature`.

Consequently, ordinary ECDSA permits fail if the delegated implementation does not implement compatible ERC-1271 validation. Delegated smart accounts that correctly implement ERC-1271 remain supported.

Recommendation: Document the requirement, add EIP-7702 compatibility tests, or implement an explicitly defined fallback policy if ECDSA signatures from delegated accounts must remain supported.

Telcoin Association: Fixed in Issue 34.

Cantina Managed: Fix verified.

3.1.5 Token pause can be bypassed through bridge burn/mint paths

Severity: Low Risk.

Context: TelcoinV3.sol#L166.

The pause check excludes mint and burn operations:

```
function _update(address from, address to, uint256 value) internal override(ERC20) {
    if (paused() && from != address(0) && to != address(0)) {
        revert EnforcedPause();
    }

    ERC20._update(from, to, value);
}
```

Therefore, if `TelcoinBridge` remains unpaused:

1. A user can burn paused-chain TEL through an outbound bridge transfer.
2. TEL can be minted to another address on a destination chain.
3. With a return route, value can ultimately be minted to another address on the originally paused chain.

Thus, pausing `TelcoinV3` does not globally freeze movement.

Recommendation: Coordinate token and bridge pausing operationally, or make bridge mint/burn honor token pause state if pause is intended as a global freeze.

Telcoin Association: Acknowledged. The token and bridge will be paused simultaneously by a third-party monitoring service.

Cantina Managed: Acknowledged.

3.1.6 `ITelcoinBridge` does not describe the deployed bridge ABI

Severity: Low Risk.

Context: ITelcoinBridge.sol#L23-L26.

The interface exposes obsolete methods:

```
function bridge(
    uint32 _dstEid,
    address _to,
    uint256 _amount,
    bytes calldata _options
) external payable returns (MessagingReceipt memory receipt);

function quote(...) external view returns (MessagingFee memory fee);

function rescueTokens(address _token, uint256 _amount) external;
```

The implementation instead exposes inherited `send()` / `quoteSend()`, and its rescue method requires a recipient:

```
function rescueTokens(
    address _token,
    uint256 _amount,
    address _to
) external onlyOwner
```

The interface also declares custom `BridgeSent` and `BridgeReceived` events that the implementation does not emit. Integrations compiled against this interface will call nonexistent selectors or monitor nonexistent events.

Recommendation: Regenerate the interface from the current implementation and add ABI-conformance tests.

Telcoin Association: Fixed in Issue 33.

Cantina Managed: Fix verified.

3.1.7 Immediate bridge replacement will block in-flight messages

Severity: Low Risk.

Context: MintBurnWrapper.sol#L95-L113.

Affected: `src/MintBurnWrapper.sol:95-102`.

A new bridge immediately replaces the existing authorized bridge:

```
function authorizeBridge(address _bridge) external onlyOwner {
    if (_bridge == address(0)) revert ZeroAddress();
    if (bridge == _bridge) revert BridgeAlreadySet();

    bridge = _bridge;
    emit BridgeAuthorized(_bridge);
}
```

After replacement, delayed inbound messages delivered through the old bridge fail when it calls `MintBurnWrapper.mint()`. LayerZero messages may be retryable, but processing remains blocked until authorization is restored or migration is otherwise coordinated.

Recommendation: Use separate inbound/outbound authorization, stage bridge replacement, and retain old inbound authority until all in-flight messages have drained.

Telcoin Association: Acknowledged. Any new bridge deployment will be paired with a new `MintBurnWrapper`, preventing authorization mismatches for in-flight messages.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 Outdated natspec in `TelcoinBridge` and `NativeBridge`

Severity: Informational.

Context: NativeBridge.sol#L63, TelcoinBridge.sol#L69, TelcoinBridge.sol#L81.

The NatSpec immediately preceding `send()` says that the function pauses the bridge:

```
/**
 * @notice Pauses the bridge — blocks send and receive.
 * @dev Overrides OFTCore.send() to enforce pausability on the standard OFT entry
 ↪  point.
 */
function send(...) external payable override whenNotPaused returns (...) {
    return _send(_sendParam, _fee, _refundAddress);
}
```

This does not represent what `send()` does. The function initiates an outbound bridge transfer and merely checks through `whenNotPaused` that the bridge has not already been paused. It neither changes pause state nor independently blocks inbound receives. The separate owner-only `pause()` function changes the pause state, while `_lzReceive()` separately applies `whenNotPaused` to inbound delivery.

The documentation also asserts that `_lzReceive()` emits a custom `BridgeReceived` event. The implementation only delegates to LayerZero and emits no such event directly:

```
/**
 * @dev Enforces pausability on inbound messages and emits BridgeReceived for
 ↪  indexing.
 */
function _lzReceive(...) internal whenNotPaused override {
    super._lzReceive(_origin, _guid, _message, _executor, _extraData);
}
```

The inherited implementation emits LayerZero's `OFTReceived`, not `BridgeReceived`.

Recommendation: Change the `send()` notice to describe an outbound transfer that reverts while paused. Update `_lzReceive()` NatSpec to reference the inherited `OFTReceived` event and document that inbound and outbound paths enforce pause state independently.

Telcoin Association: Fixed in Issue 40.

Cantina Managed: Fix verified.

3.2.2 `renounceRole()` prohibition can be bypassed through self-revocation

Severity: Informational.

Context: MigrationVault.sol#L223-L225.

The contract claims role holders cannot voluntarily give up their roles:

```
function renounceRole(bytes32, address) public pure override {
    revert CannotRenounceRole();
}
```

Nevertheless, an admin can use inherited `revokeRole()` against itself:

```
vault.revokeRole(DEFAULT_ADMIN_ROLE, msg.sender);
```

Because `DEFAULT_ADMIN_ROLE` administers itself, the sole admin can permanently disable upgrades and future role administration.

Recommendation: Override `revokeRole()` where self-revocation must be prohibited, or use `AccessControlDefaultAdminRulesUpgradeable` with delayed two-step admin transfer.

Telcoin Association: Fixed in Issue 37.

Cantina Managed: Fix verified.

3.2.3 Withdrawn legacy tokens can be recycled through a reopened migration

Severity: Informational.

Context: TokenMigration.sol#L120-L124.

Legacy tokens may be withdrawn after expiry:

```
function withdrawOldTokens(address destination) external onlyOwner {
    if (block.timestamp < migrationExpiry + withdrawalDelay) revert
    ↪  WithdrawalLocked();

    uint256 balance = oldToken.balanceOf(address(this));
    oldToken.safeTransfer(destination, balance);
}
```

The owner can subsequently move the expiry into the future:

```
function setMigrationExpiry(uint256 newMigrationExpiry) external onlyOwner {
    if (newMigrationExpiry == 0 || migrationExpiry > newMigrationExpiry) {
        revert InvalidExpiry();
    }

    migrationExpiry = newMigrationExpiry;
}
```

The withdrawn OLD tokens can then be submitted again to mint additional NEW tokens. The local `MIGRATION_SUPPLY_CAP` still prevents local `totalSupply()` from exceeding 100 billion, so the comment's claim of directly bypassing that check is imprecise. Nevertheless, recycling violates one-OLD-to-one-NEW conservation and can inflate global supply when previously minted NEW tokens have been bridged and burned locally.

Recommendation: Permanently close migration once OLD tokens are withdrawn, or prohibit expiry extension after the original expiry/first withdrawal.

Telcoin Association: Fixed in Issue 36.

Cantina Managed: `withdrawOldTokens()` now closes the migration definitively, preventing the owner from extending the expiry and recycling previously withdrawn OLD tokens to mint additional NEW tokens.

3.2.4 Unused `Nonces` import

Severity: Informational.

Context: TelcoinV3.sol#L12.

```
import {SignatureChecker} from
↪  "@openzeppelin/contracts/utils/cryptography/SignatureChecker.sol";
```

```
import {Nonces} from "@openzeppelin/contracts/utils/Nonces.sol";
```

`Nonces` is not referenced directly. Nonce handling is inherited through `ERC20Permit`.

Recommendation: Remove the unused import.

Telcoin Association: Fixed in Issue 35.

Cantina Managed: Fix verified.

3.2.5 Legacy initializer is callable because its name does not match the contract

Severity: Informational.

Context: Telcoin.sol#L56.

The contract is named `TelcoinV2`, but its constructor-style function is named `Telcoin`:

```
contract TelcoinV2 {
    function Telcoin(address _distributor) public {
        balances[_distributor] = totalSupply;
        Transfer(0x0, _distributor, totalSupply);
    }
}
```

Under Solidity 0.4.18, a constructor must match the contract name exactly. Therefore, in the current source, `Telcoin()` is a normal public runtime function. Anyone can repeatedly assign the full nominal supply to arbitrary addresses, creating effectively unbounded balances.

The existing Ethereum mainnet TEL contract is deployed with a correctly named `Telcoin` contract, so it is unaffected. Any testnet or new deployment compiled from this exact `TelcoinV2` source is unsafe.

Recommendation: Replace it with an explicit constructor-compatible name:

```
function TelcoinV2(address _distributor) public {
    balances[_distributor] = totalSupply;
    Transfer(address(0), _distributor, totalSupply);
}
```

Telcoin Association: Fixed in Issue 32.

Cantina Managed: Fix verified.

3.2.6 Supply cap is enforced independently per chain rather than globally

Severity: Informational.

Context: TelcoinV3.sol#L56.

`mint()` compares only the current chain's local `totalSupply()` against the nominal migration cap:

```
uint256 public constant MIGRATION_SUPPLY_CAP = 100_000_000_000 ether;

function mint(address to, uint256 amount) external onlyRole(MINTER_ROLE) {
    _mint(to, amount);
    if (totalSupply() > MIGRATION_SUPPLY_CAP) revert SupplyCapExceeded();
}
```

Every `TelcoinV3` deployment maintains an independent `totalSupply()`. If TEL is deployed on three chains, each deployment can therefore have up to 100 billion TEL while satisfying this check, allowing the aggregate supply across all chains to reach as much as 300 billion.

Normal LayerZero burn-and-mint bridging is intended to preserve aggregate supply, but this property is not enforced by `TelcoinV3.mint()` itself. Independent minters, migration contracts, configuration errors, or compromised mint authority can increase supply on multiple chains while every local cap check continues to pass.

Recommendation: Define whether `MIGRATION_SUPPLY_CAP` is intended as a per-chain or global limit. If global, designate a canonical supply chain or implement cross-chain supply accounting and restrict non-bridge minting on satellite chains. At minimum, monitor and reconcile the aggregate `totalSupply()` across every deployment.

Telcoin Association: Acknowledged.

Cantina Managed: Acknowledged.